

Name:- Ghulam Habib

Roll No:- 24-NTU-CS-FL-1154

Question: 1

Define array:-

An array is collection of similar data types stored in contiguous location and accessed using a index.

Example:-

```
int Marks[5] = {10, 20, 30, 50, 60}
```

2 Difference from simple var

Feature	simple variable	Array
Storage	Stores one value	Stores multiple value
Memory	single memory block	contiguous multiple blocks
access	Direct	indexed

Question No 2:-

Difference between 1D and 2D array

Feature	1D Array	2D array
Definition	Linear list of elements	Table rows and columns list of elements

Name: Ghulam Habib

Roll No: 24-NTU-CS-FL-1184

Syntax Access	int arr[5] arr[2]	int arr[3][3] arr[1][1]
visualization	1, 2, 3, 4	1 2 3 4 5 6

1D → Linear storage
2D → Grid like storage.

Question 3 :- how memory allocation

Arrays occupy continuous memory blocks so the CPU can access elements using formula.

Formula:-

$$\text{arr}[i] = \text{Base address} + (i \times \text{size of each element})$$

Example:-

array[5] = {10, 20, 30, 30}

This makes fast access is $O(1)$

Name: Chulam Habib
Roll NO: 24-NTU-CS-FL-1154

Question 4:- syntax of arrays

1D Array syntax:-

```
int marks[5]
```

```
int marks[5] = {10, 20, 30, 40}
```

2D array syntax:-

```
int marks[3][3];
```

```
int marks[3][3] = { {1, 2, 3},  
                    {4, 5, 6},  
                    {7, 8, 9} }
```

Question 5 :- what is pointer?

A Pointer is variable that stores memory address of another elements

```
int X = 10
```

```
int *Ptr = &X
```

Accessing array element with pointers

```
int arr[3] = {10, 20, 30}
```

```
int *P = arr;
```

```
cout << * (P+0)
```

```
cout << * (P+1); 20
```

Name: Ghulam Habib
Roll No: 24-NTU-CS-FL-1184

Question 6:-

In C++ array name behave like a pointer to its first element

```
int arr[] = {5, 4, 10, 50}
```

```
cout << arr;
```

```
cout << *arr;
```

```
cout << *(arr + 2);
```

Key relationship:-

$$\text{arr}[i] = *(arr + i)$$

Explanation:-

- arr gives the address of first element.
- arr + i moves the pointer i elements ahead.
- *(arr + i) access the value

Question 7:-

- Dynamic memory allocation is the process of allocating memory at runtime instead of compile time. It allows flexible memory usage base on program needs.

List:-

malloc(), realloc(), calloc(), free()

Name: Ghulam Habib
RollNo: 24-NTU-CS-FL-1154

Question 8

Dynamic memory allocation related functions:-

`malloc()` → allocate single block of memory

`calloc()` → Allocates multiple and initialize the to zero

`realloc()` → Resize previously allocated memory

`free()` free the allocated memory

```
int *ptr = (int *) malloc(5 * sizeof(int))
```

Question 9:-

insertion mean adding a new element at a specific position.

insertion at beginning:-

- shift all elements to right
- insert new value

```
for(int i = n; i > 0; i--)
```

```
arr[i] = arr[i-1]
```

```
arr[0] = value
```

insertion between:-

shift element to right by one position

Name: Ghulam Habib
Roll No: 24-NTU-CS-FL-1184

```
for (int i = n; i > pos; i--)
```

```
arr[i] = arr[i-1]
```

```
arr[pos] = newvalue
```

insertion at end

```
simple arr[n]
```

```
arr[n] = value
```

```
n++
```

Question 10:-

Deletion means removing an element and shifting remaining ones to fill the gap.

a) Deletion from beginning:-

Moves all element to ~~one~~ left:

```
for (int i = 0; i < n-1; i++)
```

```
arr[i] = arr[i+1]
```

```
n--;
```

b) Deletion from in between

Shift all elements after that position to left.

```
for (int i = pos; i < n-1; i++)
```

```
arr[i] = arr[i+1]
```

```
n--;
```


Name: Ghulam Habib

Roll No: 24-NTU-CS-FL-1154

Deletion from end:-

just reducing the array size
by:

$n--;$